Titre pro Concepteur Développeur d'Application

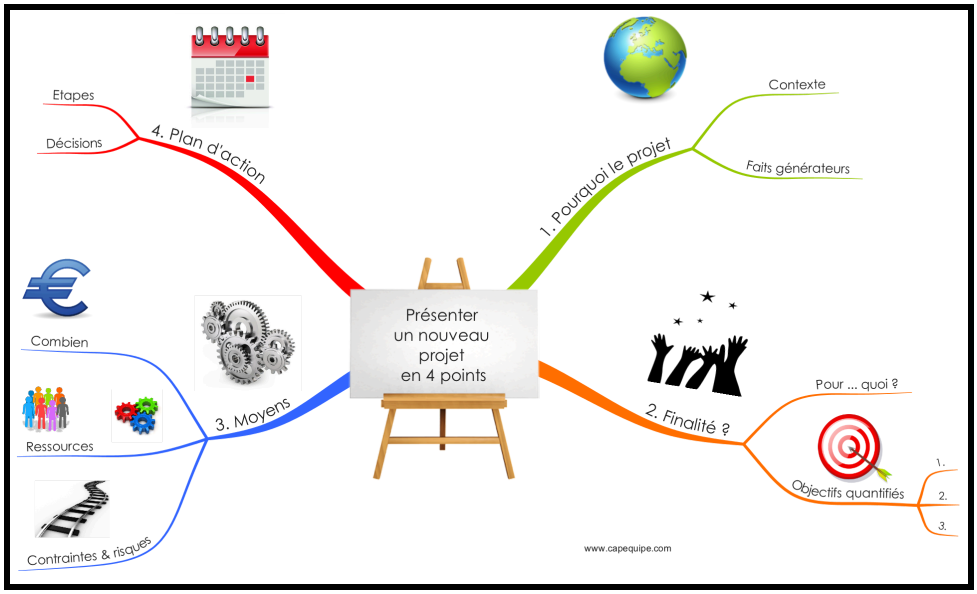
Formateurs : DESSOLY Alexandre
KERFI Outman

Tuteur : BOUTARD Benoît

Sommaire

Sommaire **2**
PRÉSENTATION DU PROJET **4**
ENVIRONNEMENT DE TRAVAIL **5**
Gestion de versions : 5
Conception et modélisation : 5
Looping a été utilisé pour modéliser les MCD (Modèles Conceptuels de Données) et les MLD (Modèles Logiques de Données). 6
Développement : 6
Présentation de l'Association ESCT BOXE **9**
Cahier des Charges **10**
Objectifs du Site 10
Fonctionnalités 10
Design et Accessibilité 10
Pages Principales 10
Espace Administrateur 11
Header et Footer 11
Page de Connexion 11
Charte Graphique **12**
La charte graphique de l'ESCT BOXE définit l'identité visuelle du club, en utilisant une palette de couleurs sophistiquée et dynamique : noir, gris, or et blanc. Cette combinaison de couleurs reflète à la fois la puissance et l'élégance du monde de la boxe, tout en créant une atmosphère accueillante et professionnelle. **12**
Palette de Couleurs 12
Typographie 12
Applications Visuelles 12
Liste des Compétences Mises en Œuvre dans le Cadre du Projet **13**
Gestion de Projet 13
Développement Web 13
Gestion de Base de Données 13
UX/UI Design 13
Tests et Qualité 13
Communication et Collaboration 14
Formation et Support Utilisateur 14
Gestion de Projet : Planning et Suivi, Environnement Humain, Objectifs de Qualité (avec Méthodes Agiles) **14**
Planning et Suivi 14
Environnement Humain 14
Objectifs de Qualité 15
Spécifications Fonctionnelles du Projet **16**
Espace Administrateur 16

Fonctionnalités Techniques 17
Objectifs de Performance et de Sécurité 17
Contraintes du projet 18
ARCHITECTURE LOGICIELLE DU PROJET 19
Les maquettes et enchaînement des maquettes 20
Modèle Entités-Associations (E/A) 22
Le MCD (modèle conceptuel de données). 22
Le MLD (modèle logique de données) 23
Le MPD (modèle physique de données) 24
Le script de création ou de modification de la base de données 25
1. Création de la Base de Données 25
2. Suppression des Tables Existantes 25
3. Création des Tables 26
Le diagramme du comportement des fonctionnalités de type cas d'utilisations 27
Cas d'utilisation 27
Le diagramme du détail des cas d'utilisations les plus significatifs de type diagramme de séquence. 28
Les spécifications techniques du projet, élaborées par le candidat, y compris pour la sécurité 29
1. Architecture Technique 29
a. Architecture Générale 29
b. Technologies Utilisées 29
1. Frontend 29
2. Backend 29
3. Base de Données 29
4. Gestion des Versions 30
2. Fonctionnalités Techniques 30
a. Pages Principales 30
b. API 30
3. Sécurité 30
a. Authentification et Autorisation 30
b. Protection des Données 30
c. Protection contre les Attaques 31
4. Performance et Optimisation 31
a. Optimisation des Performances 31
b. Monitoring et Analyse 31
5. Accessibilité 31
Les captures d'écran d'interfaces utilisateur et le code correspondant 32
Les extraits de code de composants métier 33



PRÉSENTATION DU PROJET

Dans le cadre de ma reconversion professionnelle, j'ai suivi la formation Concepteur et Développeur d'Applications(CDA) au centre de formation SOFIP situé à Douai afin d'obtenir mon titre professionnel.

Cette formation inclut une période de stage de quatre semaines, durant lesquelles j'ai mis en pratique les connaissances acquises au sein de l'association sportive ESCT BOXE située dans la ville de Tergnier dans le département de l'Aisne(02).

J'ai effectué ce stage en distanciel suite aux contraintes géographiques.

Après avoir échangé avec **M. BOUTARD Benoît**, le responsable du club, il est apparu que le club avait besoin d'une solution pour mieux gérer ses boxeurs, ses événements, ainsi que d'une partie: "site vitrine", pour mettre en avant l'association. Mon objectif principal était donc de concevoir et développer une application qui réponde à ces attentes spécifiques, tout en améliorant la gestion interne du club et sa visibilité en ligne.



ENVIRONNEMENT DE TRAVAIL

Pour réaliser ce projet, j'ai utilisé plusieurs technologies et outils collaboratifs afin de garantir un développement structuré et efficace.

Gestion de versions :

GitHub a été utilisé pour versionner le projet, assurant une gestion claire des différentes étapes du développement. Bien que ce projet soit individuel, GitHub a permis de suivre l'évolution du code.

Outils de gestion de projet:

Trello est un outil de gestion de projet en ligne

Conception et modélisation :

Figma a servi à la création des maquettes.

PlantUml a permis de créer les différents diagrammes conceptuels, notamment les diagrammes UML.



Looping a été utilisé pour modéliser les MCD (Modèles Conceptuels de Données) et les MLD (Modèles Logiques de Données).

Développement :



VS Code a été l'IDE principal, facilitant le développement grâce à ses nombreuses extensions et son terminal intégré.



Symfony a été le framework PHP choisi pour le back-end, permettant un développement rapide et structuré d'applications web dynamiques.



Angular, framework JavaScript pour le front-end, a été utilisé pour créer des interfaces utilisateur interactives et dynamiques, en assurant une bonne gestion des composants et des données.



Boîte à outils front-end puissante, extensible et riche en fonctionnalités, permettant de concevoir rapidement des interfaces attrayantes et adaptables à différents appareils.

Technologies Web :



HTML pour la structure des pages web.



CSS pour la mise en forme et le style des pages.



JavaScript pour ajouter des animations et améliorer l'expérience utilisateur.



PHP a été utilisé pour le développement côté serveur, facilitant l'interaction avec la base de données et le traitement des requêtes.



TypeScript, le langage utilisé avec Angular, a permis une meilleure gestion des types et une stabilité accrue du code sur le front-end.

Gestion de base de données :



MySQL, pour la gestion et le stockage des données.



PhpMyAdmin, une interface web conviviale pour gérer la base de données de manière simplifiée.



Apache est un serveur web qui facilite la gestion des bases de données dans les applications web. Bien qu'il ne gère pas directement les bases de données, Apache joue un rôle clé optimisant la performance et assurant la sécurité des applications web.



Wampserver64 a permis de faire fonctionner PhpMyAdmin en local.Cet environnement de travail a assuré un développement structuré et une gestion efficace des différentes composantes du projet.

Communication:



Discord pour la communication, les échanges de fichiers, et la fonction stream pour présenter l'avancement à distance.

Présentation de l'Association ESCT BOXE

Pendant ma période de stage, j'ai intégré l’association l'ESCT BOXE, un club dynamique dédié à la promotion de la boxe.

L'association organise des galas mettant en avant les talents de ses boxeurs, amateurs comme confirmés, dans une ambiance festive.

Elle propose également des journées découvertes ouvertes à tous, permettant à chacun de s’initier à la boxe dans un cadre convivial.

Encadrés par des entraîneurs expérimentés, les membres bénéficient d’un suivi personnalisé, que ce soit pour améliorer leur condition physique ou apprendre des techniques de combat.

Cette expérience m'a permis de mesurer l'impact de l'ESCT BOXE sur la communauté. Je recommande vivement à ceux qui s'intéressent à la boxe de rejoindre ce club enrichissant.

Cahier des Charges

Objectifs du Site

Permettre aux boxeurs de gérer leurs profils et d’accéder à des informations pertinentes.

Informer les utilisateurs sur les actualités du club.

Offrir une plateforme d'interaction par le biais de commentaires sur les actualités.

Assurer une gestion efficace des boxeurs et des actualités par l'administrateur.

Fonctionnalités

Design et Accessibilité

Le site sera responsive, accessible sur ordinateurs, tablettes et smartphones.

L’interface sera conviviale et rapide, offrant une expérience utilisateur optimale.

Pages Principales

Page d'Accueil

Présentation du club et de ses valeurs.

Mise en avant des actualités récentes.

Page Inscription 2024/2025

Liens pour différents documents d’inscription pour les nouveaux boxeurs.

Informations sur les modalités d’inscription.

Page Actualités

Liste des actualités avec possibilité pour les utilisateurs de commenter.

Page Profil

Espace personnel pour chaque boxeur et utilisateur.

Gestion des informations personnelles et des performances.

Historique des combats et statistiques.

Page Contact

Formulaire de contact pour les demandes générales.

Informations de contact (adresse email, numéro de téléphone, adresse postale).

Espace Administrateur

Gestion des Boxeurs

Ajouter, modifier ou supprimer des profils de boxeurs.

Consultation des statistiques et des performances des boxeurs.

Gestion des Actualités

Ajouter, modifier ou supprimer des articles d’actualité.

Modération des commentaires laissés par les utilisateurs.

Header et Footer

Header

Logo du club.

Liens vers les pages principales (Accueil, Inscription, Actualités, Contact, Profil).

Bouton d'accès à la page de connexion.

Footer

Liens de rappel vers les différentes pages.

Liens vers les réseaux sociaux du club.

Informations de contact et horaires.

Conditions Générales d’Utilisation

Page de Connexion

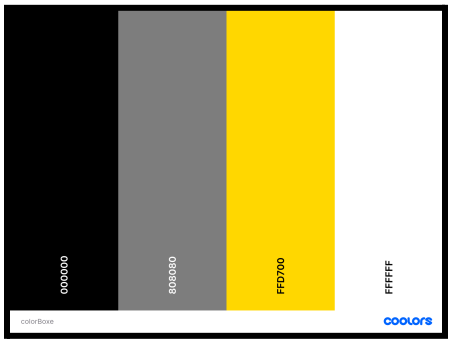
Accès réservé à l’administrateur pour gérer les boxeurs et les actualités.

Redirection vers le Dashboard d’administration après une connexion réussie.

Charte Graphique

La charte graphique de l'ESCT BOXE définit l'identité visuelle du club, en utilisant une palette de couleurs sophistiquée et dynamique : noir, gris, or et blanc. Cette combinaison de couleurs reflète à la fois la puissance et l'élégance du monde de la boxe, tout en créant une atmosphère accueillante et professionnelle.

Palette de Couleurs



Noir (#000000): **Symbolique** : Représente la force, le sérieux et la détermination.

Utilisation : Couleur principale pour les titres, les éléments graphiques marquants et les fonds de certains visuels.

Gris (#808080): **Symbolique** : Évoque la stabilité, la modernité et le professionnalisme.

Utilisation : Utilisé pour les textes secondaires, les arrière-plans et les éléments de séparation. Il apportera une touche de sophistication et de neutralité.

Or (#FFD700): **Symbolique** : Représente la victoire, l'excellence et la réussite.

Utilisation : Accentuation des éléments clés comme les boutons d'action, les bordures et les icônes. Cette couleur doit être utilisée avec parcimonie pour créer un impact visuel fort.

Blanc (#FFFFFF): **Symbolique** : Évoque la pureté, la simplicité et l'ouverture.

Utilisation : Arrière-plans, espaces vides et pour assurer une bonne lisibilité des textes. Le blanc équilibre les autres couleurs et améliore la clarté de l'information.

Typographie

La typographie utilisée dans les communications de l'association sera moderne et lisible. Une police sans-serif sera privilégiée pour les titres et les éléments accrocheurs, tandis qu'une police plus classique pourra être utilisée pour le corps de texte.

Applications Visuelles

Logo : Le logo de l'ESCT BOXE est déjà existant.

Éléments Graphiques : Des éléments graphiques comme des lignes ou des formes géométriques peuvent être utilisés en gris ou en or pour renforcer la structure visuelle des documents.

Liste des Compétences Mises en Œuvre dans le Cadre du Projet

Gestion de Projet

Planification des différentes étapes du projet.

Mise en place d'un canal discord pour échanges et partages avec le tuteur.

Suivi de l'avancement et respect des délais.

Développement Web

Conception et développement de l'interface utilisateur.

Utilisation de langages de programmation tels que HTML, CSS, PHP et JavaScript.

Mise en place de fonctionnalités dynamiques et interactives.

Gestion de Base de Données

Conception de la structure de la base de données pour stocker les informations des boxeurs et des actualités.

Écriture de requêtes mySQL pour la gestion et l'extraction des données.

UX/UI Design

Création d'une interface utilisateur intuitive et esthétique.

Réalisation de maquettes et prototypes pour tester l'expérience utilisateur.

Application des principes de design responsive pour assurer l’accessibilité sur différents appareils.

Tests et Qualité

Élaboration et exécution de scénarios de test pour valider les fonctionnalités du site.

Identification et correction des bugs et des problèmes de performance.

Communication et Collaboration

Échanges réguliers avec les parties prenantes pour recueillir des retours et ajuster le projet.

Rédaction de documents de spécifications et de rapports d’avancement.

Formation et Support Utilisateur

Création de guides et de tutoriels pour aider les utilisateurs à naviguer sur le site.

Mise en place d’un support technique pour répondre aux questions des utilisateurs.

Gestion de Projet : Planning et Suivi, Environnement Humain, Objectifs de Qualité (avec Méthodes Agiles)

Planning et Suivi

La gestion de projet pour la création du site a débuté par une planification des différentes tâches à effectuer, intégrant des méthodes agiles. Un calendrier a été élaboré, identifiant les principales étapes du projet, de la conception initiale à la mise en ligne finale. Chaque phase a été divisée en itérations, avec des délais précis pour garantir un avancement régulier.

L’outil de gestion de projet Trello, a été utilisé pour centraliser les informations et assurer une transparence dans le suivi des tâches.

Environnement Humain

L’environnement humain du projet a été essentiel à sa réussite, bien que j’aie travaillé en autonomie, en relation directe avec mon tuteur de stage. Cette collaboration a permis de bénéficier de conseils et de retours constructifs tout au long du processus de développement.

Mon tuteur a joué un rôle clé en guidant les décisions stratégiques et en fournissant des orientations sur les aspects techniques et fonctionnels du projet. Des échanges réguliers ont été organisés pour discuter des progrès, des défis rencontrés et des solutions possibles.

Bien que travaillant seul, j'ai veillé à rester ouvert aux retours et aux suggestions, ce qui a favorisé une dynamique d'amélioration continue. Les échanges avec mon tuteur ont permis de valider les choix effectués et d'ajuster le projet en fonction des besoins identifiés. Cette approche a garanti que le produit final soit en adéquation avec les attentes de l'ESCT BOXE.

Objectifs de Qualité

Les objectifs de qualité ont été définis dès le début du projet, avec un accent particulier sur la convivialité, la performance et la sécurité du site. Des critères de qualité précis ont été établis pour chaque fonctionnalité, garantissant que le produit final réponde aux attentes des utilisateurs.

Des tests rigoureux ont été réalisés à chaque étape de développement, avec des tests automatisés intégrés dans le processus de CI/CD (Intégration Continue/Déploiement Continu). Cela inclut des tests fonctionnels, des tests de performance et des tests d’ergonomie. Le retour utilisateur lors des phases de test ont également été intégrés dans les itérations suivantes pour améliorer continuellement le site.

La gestion de projet a été une démarche intégrée, alliant planning méthodique, collaboration humaine dynamique et objectifs de qualité. L'approche agile a permis une flexibilité et une réactivité accrues, essentielles pour créer une plateforme performante et conviviale, répondant aux besoins des boxeurs et des utilisateurs.

Spécifications Fonctionnelles du Projet

Les spécifications fonctionnelles décrivent les fonctionnalités et les exigences du site. Elles ont été élaborées pour garantir que celui-ci réponde aux besoins des utilisateurs tout en respectant les objectifs du projet.

Pages Principales

- **Page d'Accueil**

  Présentation du club avec un “carrousel” de photos.

  Mise en avant des dernières actualités.

  Liens directs vers les sections Inscription, Actualités, et Contact.

Page Inscription 2024/2025

Documents d'inscription pour les nouveaux membres.

Page Actualités

Liste des articles récents .

Fonctionnalité de commentaire sous chaque article, avec modération par l'administrateur.

Page Profil

Espace personnel pour chaque boxeur, affichant les informations de profil (photo, nom, performances, etc.).

Possibilité de mettre à jour ses informations personnelles.

Historique des combats et statistiques de performance.

Page Contact

Formulaire de contact pour les demandes générales.

Informations de contact (adresse email, numéro de téléphone, adresse postale).

Carte intégrée pour localiser le club.

Espace Administrateur

- **Gestion des Boxeurs**

  Interface pour ajouter, modifier ou supprimer des profils de boxeurs.

Consultation des statistiques de chaque boxeur.

- **Gestion des Actualités**

  Possibilité d'ajouter, modifier ou supprimer des articles.

  Outil de modération pour les commentaires des utilisateurs.

Fonctionnalités Techniques

- **Authentification des Utilisateurs**

  Système de connexion sécurisé pour les utilisateurs et l'administrateur.

  Récupération de mot de passe en cas d'oubli.

- **Base de Données**

  Structure de base de données pour stocker les informations des boxeurs, des actualités et des commentaires.

  Requêtes mySQL pour la gestion des données.

- **Design Responsif**

  Interface adaptée pour une utilisation sur ordinateurs, tablettes et smartphones.

  Respect des normes d'accessibilité pour tous les utilisateurs.

Objectifs de Performance et de Sécurité

- **Performance**

  Chargement rapide des pages (moins de 3 secondes).

  Optimisation des images et des ressources pour une meilleure expérience utilisateur.

- **Sécurité**

  Protection des données personnelles des utilisateurs.

  Mise en place de mesures de sécurité contre les attaques courantes (injection SQL, XSS, etc.).

Contraintes du projet

Budget : Limite des ressources financières allouées pour le développement du site (hébergement, design, développement, maintenance).

Délai : Échéance pour le lancement du site, notamment en fonction des événements sportifs ou des inscriptions de saison.

Ressources humaines : Disponibilité pour travailler sur le projet .

Technologie : Contraintes liées aux outils et plateformes utilisés (intégration avec des systèmes existants, compatibilité mobile).

Contenu : Nécessité de collecter et de créer du contenu (textes, images, vidéos) en adéquation avec l'identité du club et les attentes des utilisateurs.

Conformité : Respect des réglementations sur la protection des données (RGPD) et autres normes applicables aux sites web sportifs.

Accessibilité : Assurer que le site soit accessible à tous, y compris aux personnes en situation de handicap.

Livrables attendus

Cahier des charges : Document détaillant les besoins et objectifs du site, ainsi que les fonctionnalités souhaitées.

Maquettes et prototypes : Designs visuels du site (wireframes, maquettes graphiques) pour validation avant développement.

Site web : Le site final, incluant toutes les pages, fonctionnalités (inscriptions, actualités, contact, etc.) et intégrations nécessaires.

Documentation technique : Guides d'utilisation et documentation pour l'administration du site (gestion des contenus, mises à jour, etc.).

Stratégie de contenu : Plan de création et de mise à jour du contenu, incluant les textes, images et vidéos.

Formation : Sessions pour former les responsables du club à l'utilisation et à la gestion du site.

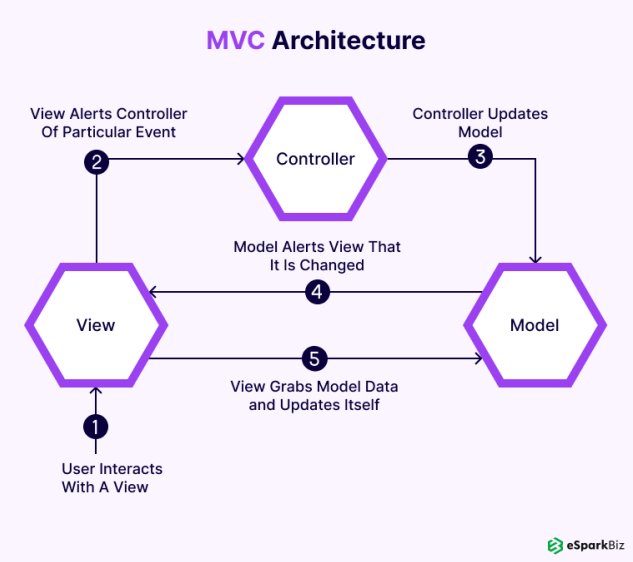
Rapport de lancement : Évaluation post-lancement des performances du site, retour des utilisateurs et ajustements nécessaires.

ARCHITECTURE LOGICIELLE DU PROJET

Le projet est construit en utilisant les frameworks Symfony et Angular, qui reposent sur l'architecture logicielle Modèle-Vue-Contrôleur (MVC). Cette architecture se divise en trois composants aux responsabilités distinctes, chacun étant optimisé par les spécificités des frameworks utilisés :

- **Modèle (Model)** : Dans Symfony, le modèle représente la logique de l'application et les données. Il gère l'accès aux données via des entités et des repositories, facilitant la récupération, la mise à jour et la validation des informations à l'aide de Doctrine ORM. Le modèle notifie également la vue de tout changement, garantissant que les données affichées sont toujours à jour.
- **Vue (View)** : Avec Angular, la vue constitue l'interface utilisateur dynamique de l'application. Elle utilise des composants pour afficher les données provenant du modèle et intégrer des éléments interactifs. Angular facilite également la gestion des événements et des interactions utilisateur, en envoyant ces interactions au contrôleur pour traitement. La vue est donc responsable de la présentation et de l'affichage des données, offrant une expérience utilisateur engageante.
- **Contrôleur (Controller)** : Dans Symfony, le contrôleur fait le lien entre le modèle et la vue. Il traite les entrées de l'utilisateur, effectue des opérations sur le modèle et met à jour la vue en conséquence. Angular, quant à lui, utilise des services et des contrôleurs pour gérer la logique de l'application côté client, orchestrant ainsi l'interaction entre les composants de l'interface. Le contrôleur gère donc la logique de l'application et les flux de données entre les modèles et les vues.

En séparant ces trois composants, l'architecture MVC facilite la maintenance, l'évolutivité et la testabilité des applications. Grâce à l'utilisation de Symfony pour le backend et d'Angular pour le frontend, cette structure robuste permet de développer des applications web performantes, bien organisées et réactives, tout en assurant une gestion efficace des interactions et des données. Cette approche permet également d'exploiter pleinement les fonctionnalités de chaque framework, garantissant ainsi une expérience utilisateur fluide et dynamique.



Les maquettes et enchaînement des maquettes



1. Maquette de l'Accueil

Éléments principaux :

Bannière principale : Image accrocheuse avec le logo du club et un slogan inspirant.

Menu de navigation : Liens vers les sections (Accueil, Actualités, Inscription 2024/2025, Contact, Se connecter/S'inscrire, Profil).

Section d'actualités : Dernières nouvelles ou événements à venir.

Bouton d'inscription rapide : Mise en avant pour attirer de nouveaux membres.

2. Maquette des Actualités

Éléments principaux :

Liste d’articles : Titre, date, extrait d’actualité avec images.

Article individuel : Page dédiée avec contenu complet, images et possibilité de commenter.

Filtrage par catégorie : Options pour naviguer par type d'actualité (événements, résultats, annonces).

Partage sur les réseaux sociaux : Boutons de partage pour chaque article.

3. Maquette d'Inscriptions 2024/2025

Éléments principaux :

Liens d'inscription : Accès facile au formulaire d'inscription.

Documents relatifs à l’inscription : Liens pour télécharger la demande de licence et le certificat médical.

4. Maquette de la Page de Contact

Éléments principaux :

Formulaire de contact : Champs pour nom, email, sujet et message.

Coordonnées du club : Adresse, téléphone, email.

Carte interactive : Localisation du club.

Heures d'ouverture : Affichage des heures d'ouverture pour les visites.

5. Maquette de Se Connecter/S'inscrire

Éléments principaux :

Formulaire de connexion : Champs pour email et mot de passe, avec un lien pour réinitialiser le mot de passe.

Formulaire d'inscription : Champs pour les informations nécessaires (nom, email, âge, etc.).

Informations sur les avantages de l’inscription : Brève description des bénéfices d'être membre.

6. Maquette du Profil

Éléments principaux :

Mon Profil : Affichage des informations personnelles avec possibilité de modification.

Ma Messagerie : Accès à la boîte de réception pour les messages des entraîneurs ou des membres.

Login : Informations de connexion actuelles avec un lien pour changer le mot de passe.

Déconnexion : Bouton clair pour se déconnecter du compte.

Boxeurs : Liste des boxeurs associés au profil, avec détails sur chaque boxeur (nom, catégories, performances).

Enchaînement des Maquettes

L'enchaînement des maquettes doit refléter un parcours utilisateur fluide :

Accueil → Découverte du club et des actualités.

Actualités → Exploration des dernières nouvelles.

Inscriptions 2024/2025 → Si intéressé, navigation vers le formulaire d'inscription.

Contact → En cas de questions ou besoin d’informations supplémentaires.

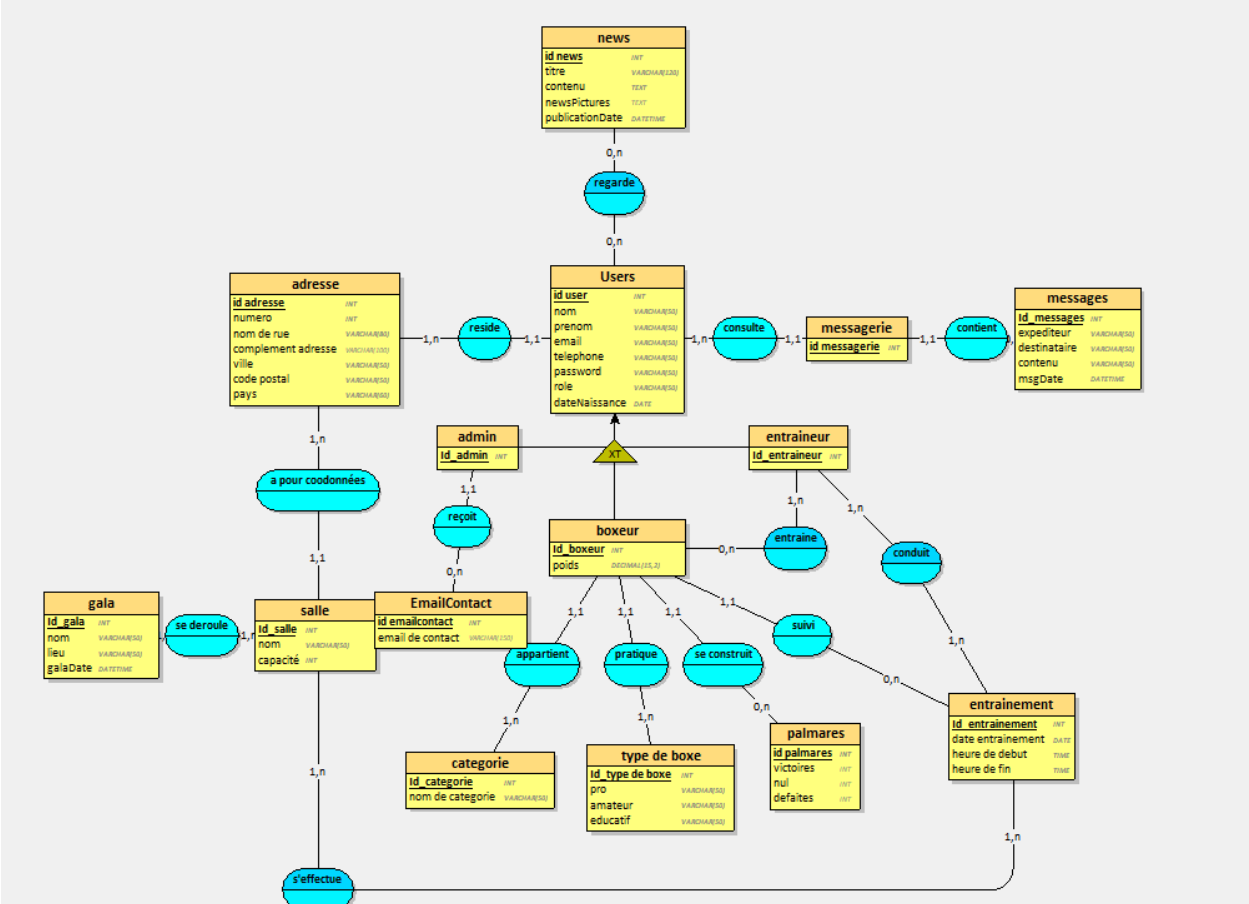
Se Connecter/S'inscrire → Accès à la connexion ou à l'inscription.

Profil → Consultation et gestion des informations personnelles.

Modèle Entités-Associations

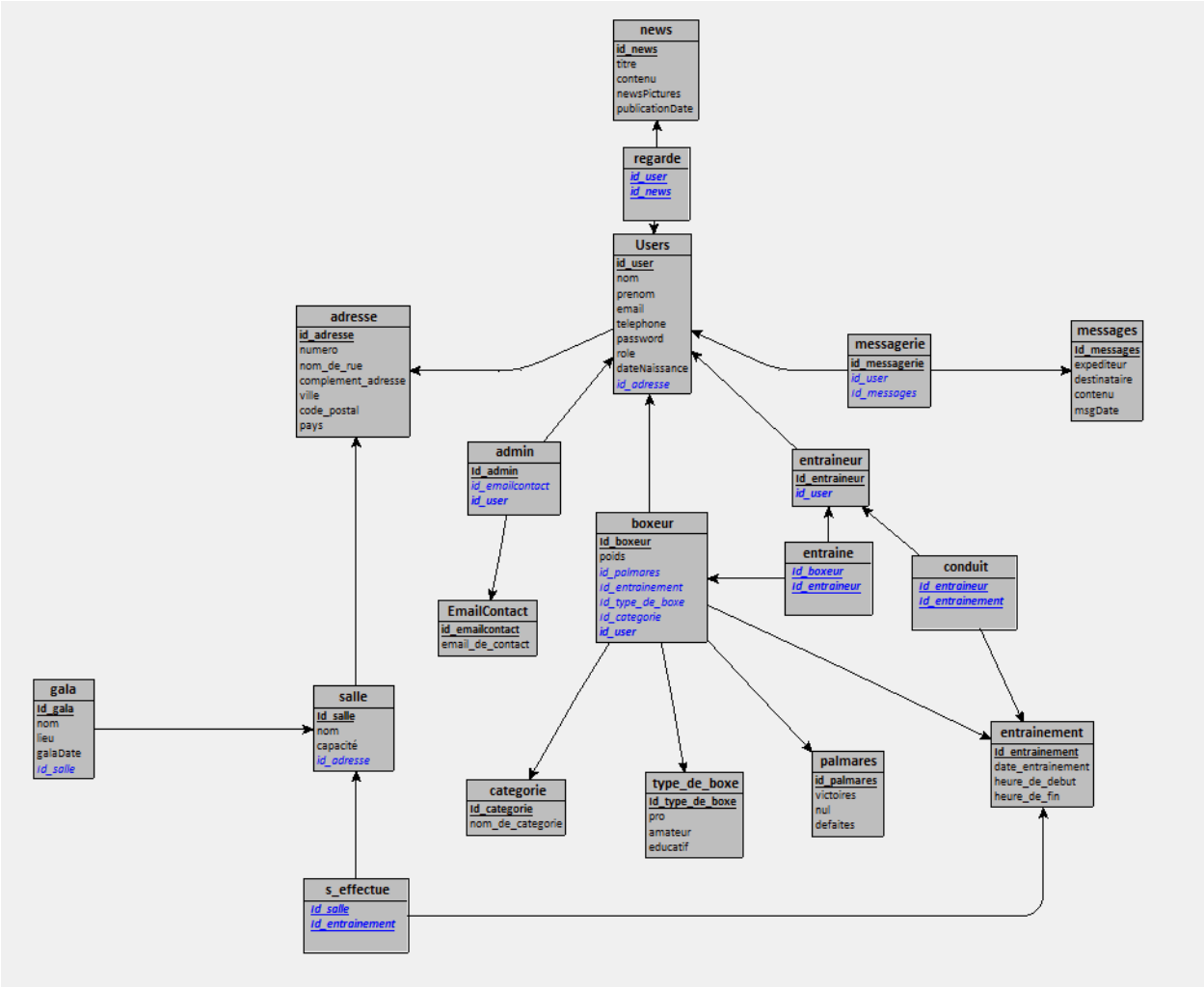
Le MCD (modèle conceptuel de données).

Le Modèle Conceptuel de Données (MCD) est un outil essentiel pour la conception des bases de données. Il représente graphiquement les entités d’un système d’information, comme "Boxeur" ou "Messagerie", ainsi que leurs attributs et relations. Par exemple, un boxeur peut participer à plusieurs combats, tandis qu'une messagerie peut être utilisée par plusieurs utilisateurs. Le MCD sert de base pour développer le Modèle Logique de Données (MLD) et le Modèle Physique de Données (MPD), facilitant ainsi la communication entre les parties prenantes et assurant une structure claire des données.



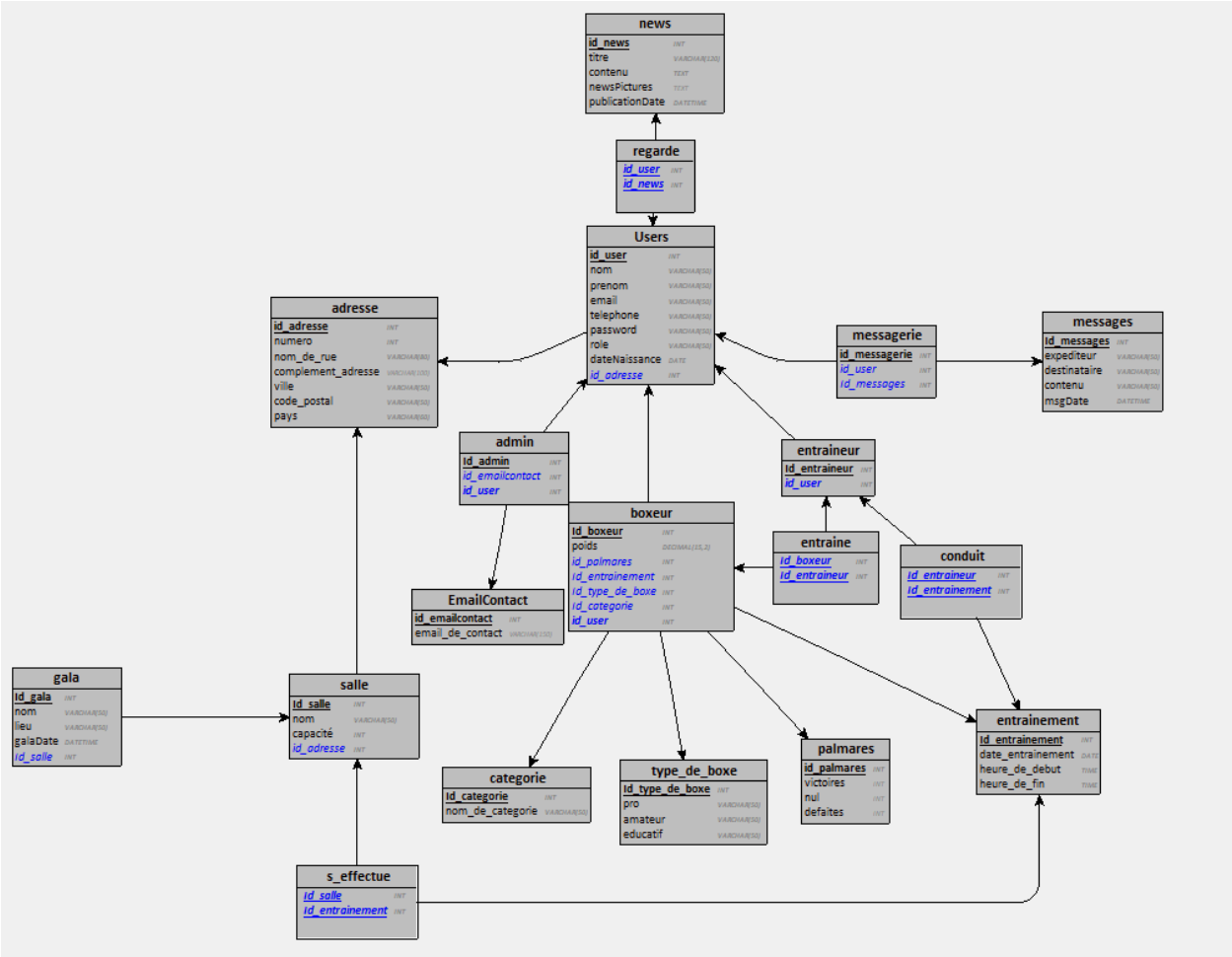
Le MLD (modèle logique de données)

Le Modèle Logique de Données (MLD) est une étape clé dans la conception des bases de données. Il traduit le Modèle Conceptuel de Données (MCD) en une structure plus détaillée, définissant les tables, les clés primaires et les relations entre elles. Par exemple, une table "Boxeur" peut contenir des attributs comme "Nom" et "Poids", tandis qu'une table "Messagerie" peut inclure des informations sur les utilisateurs et les messages échangés. Le MLD facilite la mise en œuvre du Modèle Physique de Données (MPD) et assure une organisation efficace des données.



Le MPD (modèle physique de données)

Le Modèle Physique de Données (MPD) est la dernière étape dans la conception des bases de données. Il traduit le Modèle Logique de Données (MLD) en une structure spécifique à un système de gestion de bases de données, en précisant le type de données et les index. Par exemple, la table "Boxeur" pourrait être créée avec des colonnes pour "Nom" et "Poids", tandis que la table "Messagerie" pourrait inclure des colonnes pour "Utilisateur" et "Contenu". Le MPD optimise les performances et garantit une gestion efficace des données lors de l'implémentation.



Le script de création ou de modification de la base de données

(liste des principales commandes SQL en annexe)

Il est possible d'exécuter les commandes SQL directement sous symfony en utilisant Doctrine.

Doctrine est un ORM (Object-Relational Mapping) pour PHP, souvent utilisé avec Symfony pour simplifier la gestion des bases de données. Voici un bref résumé de ses principales caractéristiques :

Doctrine permet ainsi de travailler avec des objets au lieu de SQL, rendant le code plus propre et plus facile à maintenir dans les applications Symfony.

1. Création de la Base de Données

version SQL:

CREATE DATABASE IF NOT EXISTS esct_boxe;
USE esct_boxe;



version Symfony:

```
PS C:\Users\Admin\Desktop\siteEsctBoxe\backBoxe>php bin/console doctrine:database:create
```

Pour cela il faut configurer l'adresse de la base de données dans le fichier ".env.local"

```
DATABASE_URL="mysql://root:@127.0.0.1:3306/esctBoxe?serverVersion=8.0.32&charset=utf8mb4"
```

2. Suppression de la Base de Données

version SQL:

DROP DATABASE esctboxe



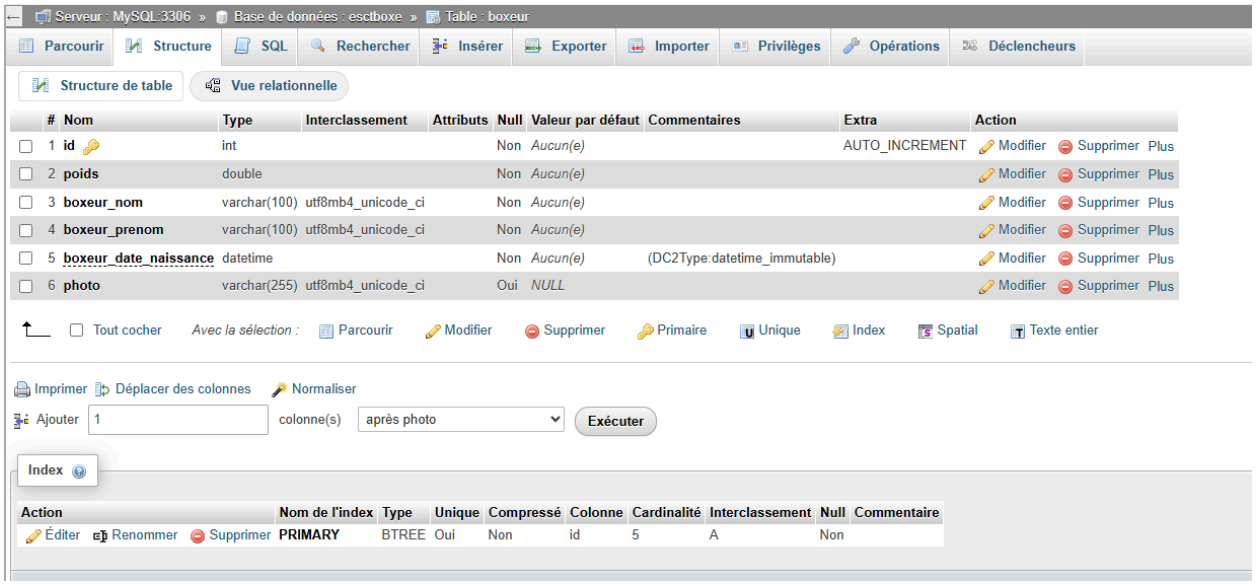
version Symfony:

```
PS C:\Users\Admin\Desktop\siteEsctBoxe\backBoxe>php bin/console doctrine:database:drop
```

3. Création des Tables

version SQL:

CREATE TABLE Boxeur (id INT AUTO_INCREMENT PRIMARY KEY,
poids DOUBLE NOT NULL,
boxeur_nom VARCHAR(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
boxeur_prenom VARCHAR(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
boxeur_date_naissance DATETIME NOT NULL,
photo VARCHAR(255) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT NULL);

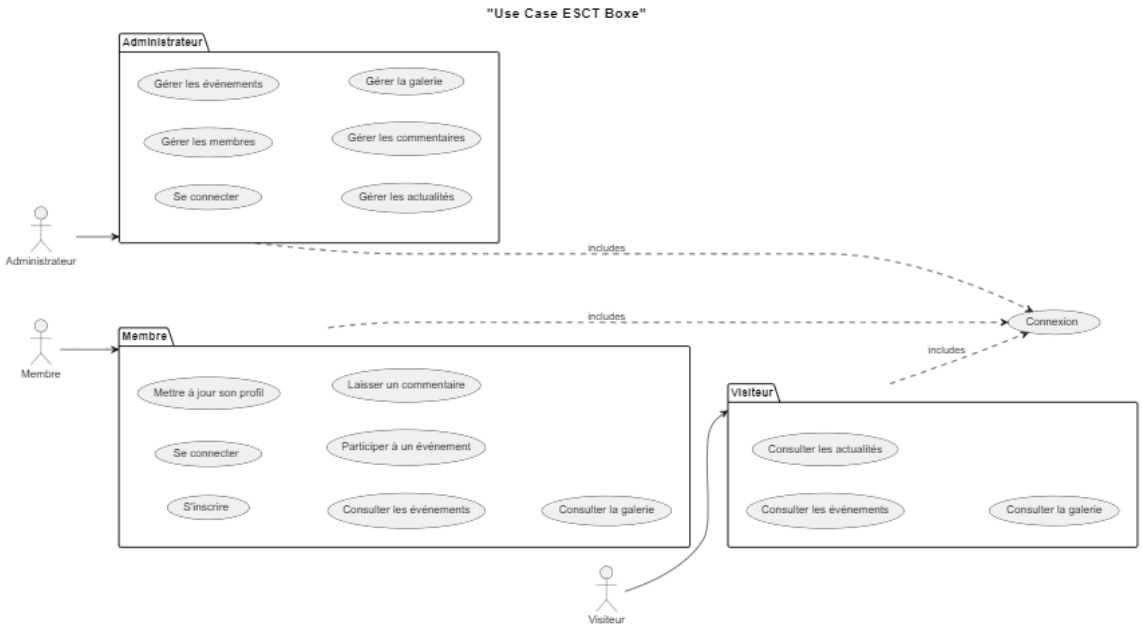


version Symfony (détails en annexe):

```
PS C:\Users\Admin\Desktop\siteEsctBoxe\backBoxe> php bin/console make:entity
```

Le diagramme du comportement des fonctionnalités de type cas d'utilisations

Schéma:



Cas d'utilisation

1. **Pour le Membre** :

   S'inscrire sur le site, Se connecter, Mettre à jour son profil, Consulter les événements, Participer à un événement, Laisser un commentaire sur une actualité, Consulter la galerie

2. **Pour l'Administrateur** :

   Se connecter, Gérer les membres (ajouter, modifier, supprimer), Gérer les événements (ajouter, modifier, supprimer), Gérer les actualités (ajouter, modifier, supprimer), Gérer les commentaires (modérer, supprimer), Gérer les images dans la galerie

3. **Pour le Visiteur** :

   Consulter les événements, Consulter les actualités, Consulter la galerie

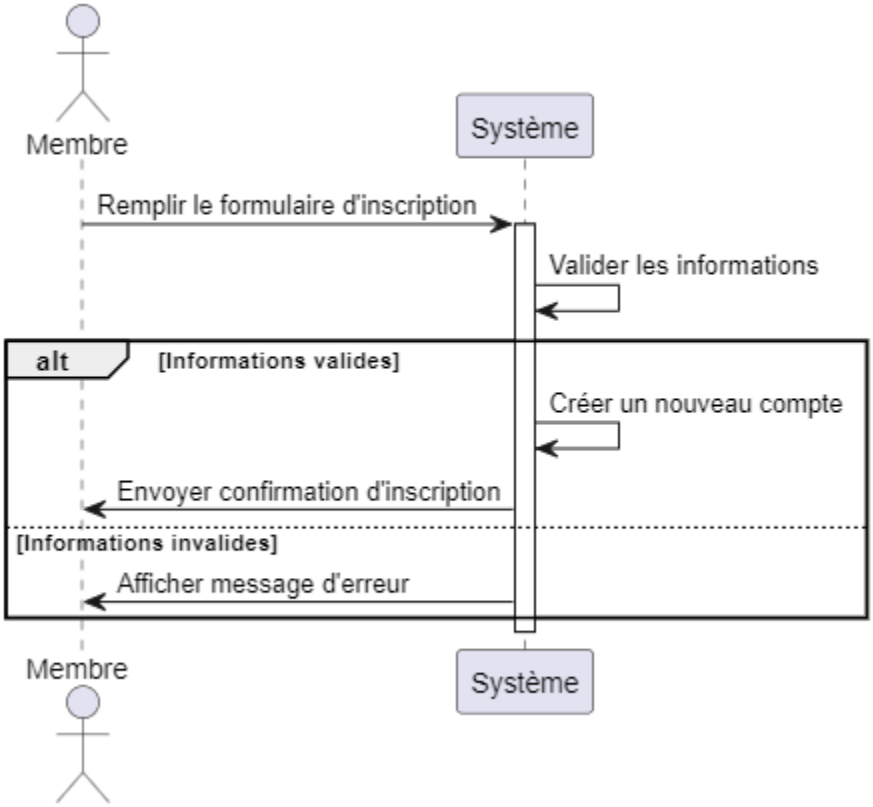
Le diagramme du détail des cas d’utilisations les plus significatifs de type diagramme de séquence.

Ce diagramme de séquence illustre le processus d'inscription d'un membre dans le système. Il montre l'interaction entre l'acteur (Membre) et le système au cours de ce processus.

1. Remplissage du formulaire : L'utilisateur remplit un formulaire d'inscription.
2. Validation des informations : Le système valide les informations fournies.
3. Scénarios possibles :
   - Informations valides : Si les données sont correctes, un nouveau compte est créé et l'utilisateur reçoit une confirmation de son inscription.

- Informations invalides : Si les données ne sont pas conformes, un message d'erreur est affiché pour informer l'utilisateur du problème.

Ce diagramme aide à visualiser les étapes et les décisions prises durant l'inscription, facilitant ainsi la compréhension du flux d'interactions.



Les spécifications techniques du projet,, y compris pour la sécurité

1. Architecture Technique

a. Architecture Générale

Client : Interface utilisateur accessible via des navigateurs web, avec un design responsive pour mobiles et tablettes.

Serveur : Application web utilisant le framework **Symfony** pour gérer la logique métier.

Base de Données : Système de gestion de base de données (SGBD) **MySQL** pour stocker les informations.

b. Technologies Utilisées

1. Frontend

Technologies : Utilisation de **HTML5**, **CSS3**, et **JavaScript**(et **TypeScript** pour angular)

Framework : **Angular** est intégré pour offrir une expérience utilisateur dynamique et réactive. Ce framework permet de créer des applications monopages (SPA), facilitant ainsi une navigation fluide et une interaction rapide. Son système de composants rend le développement plus modulable et maintenable.

2. Backend

Langage : Développement en **PHP**.

Framework : Utilisation de **Symfony** pour une gestion efficace de la logique métier et des interactions avec la base de données. Symfony facilite la création d'API RESTful, la gestion des routes, et l'implémentation de la sécurité, optimisant ainsi le développement et la performance.

3. Base de Données

SGBD : Utilisation de **MySQL** pour stocker toutes les données nécessaires, comme les informations des membres, les événements et les actualités. MySQL offre robustesse et flexibilité pour la gestion des données.

4. Gestion des Versions

Outil : Utilisation de **Git** pour le suivi des modifications de code. Cela permet une collaboration efficace entre les développeurs et assure une gestion organisée des versions de l'application.

2. Fonctionnalités Techniques

a. Pages Principales

Page d'Accueil : Présentation du club, mise en avant des actualités et liens vers les sections clés.

Page Inscription : Formulaire d'inscription avec validation des données et confirmation par email.

Page Actualités : Liste des articles avec possibilité de filtrer par catégorie et fonctionnalité de commentaire.

Espace Membre : Profil avec informations personnelles, historique et statistiques de performance.

Espace Administrateur : Gestion des membres et des actualités, modération des commentaires.

b. API

Mise en place d'une **API RESTful** pour gérer les opérations CRUD (Créer, Lire, Mettre à jour, Supprimer) sur les entités (membres, événements, actualités).

3. Sécurité

a. Authentification et Autorisation

Système d'authentification : Utilisation de **JSON Web Tokens (JWT)** pour les sessions utilisateurs.

Gestion des rôles : Mise en place d'un système de gestion des rôles pour limiter l'accès à certaines fonctionnalités.

b. Protection des Données

Chiffrement des mots de passe : Utilisation de **password hasher** pour stocker les mots de passe des utilisateurs de manière sécurisée.

Chiffrement des données sensibles : Chiffrement des informations sensibles dans la base de données, comme les adresses email et les numéros de téléphone.

c. Protection contre les Attaques

Prévention des injections SQL : Utilisation de requêtes préparées pour toutes les interactions avec la base de données.

Protection contre les attaques XSS : Validation et échappement des entrées utilisateur.

Protection CSRF : Mise en place de jetons CSRF pour les formulaires sensibles.

4. Performance et Optimisation

a. Optimisation des Performances

Chargement rapide des pages : Optimisation des images et des fichiers JavaScript/CSS pour réduire le temps de chargement (ciblé en moins de 3 secondes). Utilisation de formats d’images appropriés (JPEG pour les photos, PNG pour les images, et WEBP pour une meilleure compression).

Mise en cache : Utilisation de mécanismes de mise en cache pour les données fréquemment consultées.

b. Monitoring et Analyse

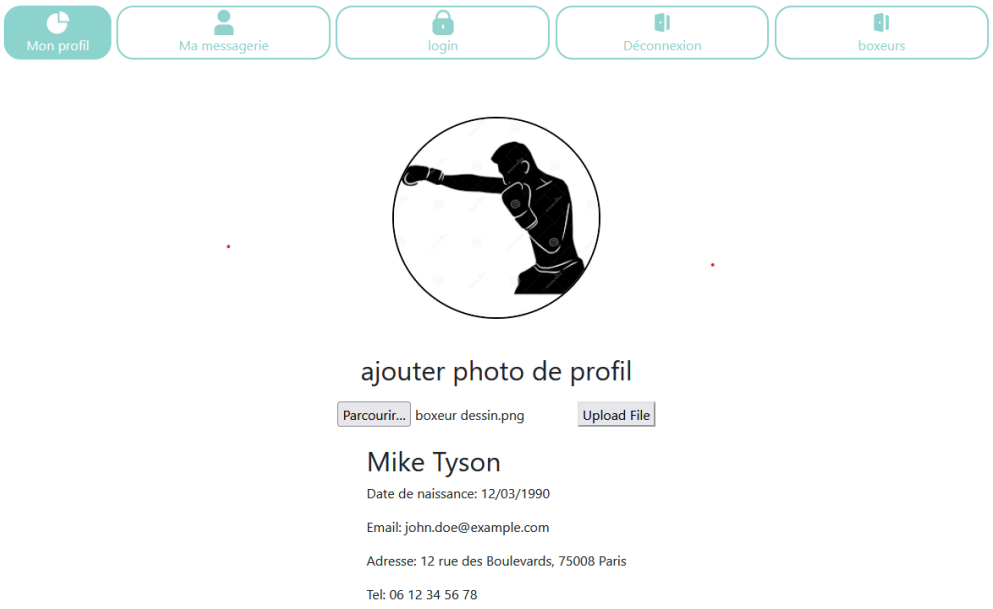
Utilisation d'outils comme **Google Analytics** pour le suivi des performances du site.

5. Accessibilité

Respect des normes **WCAG (Web Content Accessibility Guidelines)** pour garantir que le site soit accessible à tous, y compris aux personnes en situation de handicap.

Les captures d’écran d’interfaces utilisateur et le code correspondant

capture interface profil:



code html dans composant angular:

```
<div class="ajoutPhoto">
  <div class="photo">
    <img [src]="imagePreview" alt="Image de profil" />
  </div>
  <h3>ajouter photo de profil</h3>
  <div class="upload">
    <input type="file" #fileInput (change)="onFileSelected($event)" />
    <button (click)="uploadFile()">Upload File</button>
    <p *ngIf="uploadResponse">{{ uploadResponse }}</p>
  </div>
</div>

<div class="profil">
  <div class="profil-info">
    <h2>Mike Tyson</h2>
    <p>Date de naissance: 12/03/1990</p>
    <p>Email: john.doe&#64;example.com</p>
    <p>Adresse: 12 rue des Boulevards, 75008 Paris</p>
    <p>Tel: 06 12 34 56 78</p>
    <br />
  </div>
</div>
```

code type script de la fonction selectFile:



```
 // Fonction pour sélectionner une image à partir d'un événement déclenché (par exemple, lors du choix
d'un fichier dans un input de type "file")
onFileSelected(event: Event): void {
  // Récupération de l'élément input à partir de l'événement et conversion en HTMLInputElement
  const input = event.target as HTMLInputElement;

  // Vérification si des fichiers sont sélectionnés et s'il y a au moins un fichier dans la liste
  if (input.files && input.files.length > 0) {
    // Sélectionne le premier fichier dans la liste et le stocke dans la variable "selectedFile"
    this.selectedFile = input.files[0];

    // Affiche le nom du fichier sélectionné dans la console pour vérification
    console.log('Fichier sélectionné:', this.selectedFile.name);

    // Création d'une instance de FileReader pour lire le fichier et générer un aperçu
    const reader = new FileReader();

    // Lorsque la lecture est terminée, stocke le résultat (l'aperçu de l'image) dans la variable
"imagePreview"
    reader.onload = () => {
      this.imagePreview = reader.result;
    };

    // Lit le fichier sélectionné sous forme de Data URL (base64), utile pour l'aperçu de l'image dans
une balise <img>
    reader.readAsDataURL(this.selectedFile);
  }
}
```

Les extraits de code de composants métier

la présentation d’éléments de sécurité de l’application

1. Authentification et Autorisation

- **Authentification JWT** : Utilisation de JSON Web Tokens (JWT) pour authentifier les utilisateurs. Lorsqu'un utilisateur se connecte, un token est généré et envoyé au client. Ce token doit être inclus dans les en-têtes des requêtes pour accéder aux ressources protégées.
- **Gestion des rôles** : Mise en place d'un système de gestion des rôles pour limiter l'accès à certaines fonctionnalités (par exemple, seuls les administrateurs peuvent gérer les membres ou publier des actualités).

2. Protection des Données Sensibles

- **Chiffrement des mots de passe** : Utilisation de l'interface PasswordHasher dans Symfony pour stocker les mots de passe des utilisateurs de manière sécurisée. Cela garantit que même si la base de données est compromise, les mots de passe ne seront pas exposés.
- **Chiffrement des données sensibles** : Chiffrement des informations sensibles dans la base de données, comme les adresses email et les numéros de téléphone, pour protéger la vie privée des utilisateurs.

3. Sécurisation des Requêtes HTTP

- **HTTPS** : Utilisation de HTTPS pour sécuriser les communications entre le client (Angular) et le serveur (Symfony). Cela chiffre les données transmises et empêche les attaques de type "man-in-the-middle".
- **Contrôle d’accès CORS** : Configuration des en-têtes CORS (Cross-Origin Resource Sharing) dans Symfony pour contrôler quelles origines peuvent accéder aux ressources de l'API, réduisant ainsi le risque d'attaques CSRF (Cross-Site Request Forgery).

4. Validation des Données

- **Validation des entrées utilisateur** : Mise en place de validations côté serveur dans Symfony pour toutes les entrées utilisateur afin d'éviter les injections SQL et d'autres attaques basées sur l'exploitation de données non sécurisées.

5. Protection contre les Attaques Courantes

- **Protection contre les injections SQL** : Utilisation de requêtes préparées et d'ORM/ODM dans Symfony pour éviter les injections SQL.
- **Mesures de prévention XSS** : Dans Angular, échapper tous les contenus dynamiques injectés dans le HTML, en utilisant des bibliothèques comme DOMPurify.
- **Limitation des tentatives de connexion** : Mise en place de mécanismes dans Symfony pour limiter les tentatives de connexion et éviter les attaques par force brute, comme un verrouillage temporaire du compte après plusieurs tentatives échouées.

6. Surveillance et Journaux

- **Logging des erreurs et des accès** : Mise en place de systèmes de journalisation dans Symfony pour enregistrer les accès à l'application et les erreurs rencontrées. Cela permet de détecter des comportements suspects et d'améliorer la sécurité.
- **Surveillance des activités suspectes** : Utilisation d'outils de surveillance pour détecter des comportements anormaux, comme des tentatives de connexion inhabituelles ou des accès à des zones sensibles.

7. Sécurité des Routes et Interactions Angular

- **Routes sécurisées** : Configuration des routes dans Angular pour protéger l'accès aux différentes parties de l'application, en limitant l'accès selon les rôles des utilisateurs.
- **Guards** : Utilisation de guards dans Angular pour déterminer si une route peut être activée. Par exemple, un `AuthGuard` vérifie l'authentification de l'utilisateur avant d'accéder à des routes protégées.
- **Interceptors** : Implémentation d'interceptors dans Angular pour intercepter les requêtes HTTP et ajouter des en-têtes d'authentification, comme le token JWT, garantissant ainsi que toutes les communications avec le backend Symfony sont sécurisées.

Conclusion

La mise en œuvre de ces éléments de sécurité contribue à protéger l'application contre diverses menaces, à garantir la confidentialité des données des utilisateurs et à maintenir la confiance des membres du club. La sécurité doit être une priorité tout au long du développement et de la maintenance de l'application.

la présentation du plan de tests

Voici un plan de tests détaillé pour l'application de gestion du site de l'ESCT BOXE. Ce plan couvre différents types de tests pour garantir la fonctionnalité, la performance et la sécurité de l'application.

1. Objectifs des Tests

- **Vérifier la conformité des fonctionnalités** : S'assurer que toutes les fonctionnalités spécifiées sont mises en œuvre et fonctionnent comme prévu.
- **Évaluer la performance** : Tester la réactivité et la rapidité de l'application sous différentes charges.
- **Identifier et corriger les défauts** : Détecter les bugs et les failles de sécurité avant le déploiement.
- **Valider l'expérience utilisateur** : Assurer que l'interface est intuitive et que les utilisateurs peuvent naviguer facilement.

2. Types de Tests

a. Tests Fonctionnels

- **Tests des pages principales** :
  - Vérifier que toutes les pages (Accueil, Inscription, Actualités, etc.) s'affichent correctement.
  - S'assurer que les liens de navigation fonctionnent.
- **Tests des formulaires** :
  - Tester le formulaire d'inscription (validation des champs, soumission réussie).
  - Vérifier les messages d'erreur en cas de saisie invalide.
- **Tests de l'espace administrateur** :
  - Vérifier que l'administrateur peut ajouter, modifier et supprimer des membres et des actualités.
  - Tester la fonctionnalité de modération des commentaires.

b. Tests de Performance

- **Tests de charge** :

    - Simuler un grand nombre d'utilisateurs simultanés pour évaluer le comportement de l'application sous charge.
    - Mesurer le temps de réponse pour les requêtes sous différentes conditions de charge.

- **Tests de vitesse** :
    - Analyser le temps de chargement des pages pour s'assurer qu'il est inférieur à 3 secondes.

c. Tests de Sécurité

- **Tests de vulnérabilité** :
    - Effectuer des tests d'intrusion pour détecter les failles de sécurité (injections SQL, XSS, etc.).
    - Vérifier la gestion des sessions et des tokens d'authentification.
- **Tests de conformité RGPD** :
    - Vérifier que les données personnelles des utilisateurs sont correctement protégées et que les politiques de confidentialité sont respectées.

d. Tests d'Intégration

- **Tests d'API** :
    - Tester toutes les API pour s'assurer qu'elles fonctionnent correctement et retournent les réponses appropriées.
    - Vérifier les intégrations avec des services tiers (paiements, réseaux sociaux).

e. Tests d'Utilisabilité

- **Tests d'expérience utilisateur** :
    - Réaliser des sessions d'observation avec des utilisateurs réels pour évaluer l'ergonomie et la facilité de navigation.
    - Recueillir des retours sur la clarté des informations et la fluidité de l'interaction.

3. Outils de Tests

- **Selenium** : Pour les tests automatisés de l'interface utilisateur.
- **Postman** : Pour tester les API.
- **JMeter** : Pour les tests de charge et de performance.
- **OWASP ZAP** : Pour les tests de sécurité.

4. Critères de Réussite

- **Fonctionnalités** : Tous les tests fonctionnels doivent réussir sans bugs majeurs.
- **Performance** : Le site doit répondre en moins de 3 secondes sous une charge raisonnable.
- **Sécurité** : Aucun défaut critique ne doit être identifié lors des tests de sécurité.
- **Utilisabilité** : Les retours des utilisateurs doivent être majoritairement positifs.

5. Planification des Tests

- **Phase de développement** : Tests unitaires et d'intégration continus pendant le développement.
- **Phase de pré-production** : Tests fonctionnels, de performance et de sécurité avant le lancement.
- **Phase post-lancement** : Tests de maintenance réguliers pour identifier et corriger les problèmes qui apparaissent après le déploiement.

Conclusion

Ce plan de tests est essentiel pour garantir que l'application de gestion de l'ESCT BOXE soit robuste, sécurisée et répondent aux attentes des utilisateurs. En suivant ce plan, l'équipe peut s'assurer que toutes les fonctionnalités sont correctement mises en œuvre et que le site offre une expérience utilisateur fluide et sécurisée.

la présentation d’un jeu d’essai élaboré par le candidat de la fonctionnalité la plus représentative (données en entrée, données attendues, données obtenues) et analyse des écarts éventuels

Fonctionnalité : Inscription d’un Nouveau Membre

1. Description de la Fonctionnalité

Cette fonctionnalité permet à un nouvel utilisateur de s'inscrire en remplissant un formulaire avec ses informations personnelles. Le système doit valider les données saisies, enregistrer le membre dans la base de données et envoyer un email de confirmation.

---

2. Jeu d'Essai

Cas de Test : Inscription d'un Nouveau Membre

Données en Entrée	Données Attendues	Données Obtenues	Écart
Nom : "Dupont"	Inscription réussie	Inscription réussie	Aucune
Prénom : "Jean"	Email de confirmation envoyé	Email de confirmation envoyé	Aucune
Email : "jean.dupont@example.com"	ID_membre dans la base de données	ID_membre dans la base de données	Aucune
Téléphone : "0123456789"	Profil créé avec les bonnes informations	Profil créé avec les bonnes informations	Aucune
Date de Naissance : "1990-01-01"	Confirmation d'inscription affichée	Confirmation d'inscription affichée	Aucune
Type d'adhésion : "adulte"	Aucune erreur de validation	Aucune erreur de validation	Aucune

---

3. Analyse des Écarts

Cas de Test 1 : Inscription avec des données valides

- **Données en entrée** : Valides (comme indiqué ci-dessus).

- **Données obtenues** : Correspondent aux attentes.
- **Analyse** : Aucune anomalie. Le processus d'inscription fonctionne correctement.

Cas de Test 2 : Inscription avec un email invalide

Données en Entrée	Données Attendues	Données Obtenues	Écart
Nom : "Dupont" Prénom : "Jean" Email : "jean.dupont@.com" Téléphone : "0123456789" Date de Naissance : "1990-01-01" Type d'adhésion : "adulte"	Message d'erreur "Email invalide"	Message d'erreur "Email invalide"	Aucune

- **Analyse** : Le système détecte correctement l'email invalide et affiche le message d'erreur approprié. Aucune anomalie.

Cas de Test 3 : Inscription avec un champ obligatoire manquant

Données en Entrée	Données Attendues	Données Obtenues	Écart
Nom : "" Prénom : "Jean" Email : "jean.dupont@example.com"	Message d'erreur "Nom requis"	Message d'erreur "Nom requis"	Aucune

Téléphone : "0123456789"

Date de Naissance :
"1990-01-01"

Type d'adhésion : "adulte"

- **Analyse** : Le système gère correctement l'absence d'un champ obligatoire en affichant le message d'erreur adéquat. Aucune anomalie.

Le jeu d'essai démontre que la fonctionnalité d'inscription est robuste et gère correctement différents scénarios d'entrée. Les résultats obtenus correspondent aux attentes définies, et les écarts, lorsqu'ils se produisent, sont gérés efficacement par le système. Cela indique que l'application répond bien aux besoins des utilisateurs en matière d'inscription.

ANNEXES

Les principales commandes SQL utilisées dans MySQL, organisées par catégorie :

1. Création et Gestion des Bases de Données

- Créer une base de données : CREATE DATABASE nom_base;
- Afficher les bases de données : SHOW DATABASES;
- Utiliser une base de données : USE nom_base;

- Supprimer une base de données : DROP DATABASE nom_base;

2. Création et Gestion des Tables

- Créer une table : CREATE TABLE nom_table (colonne1 type1, colonne2 type2, ...);
- Afficher les tables : SHOW TABLES;
- Décrire une table : DESCRIBE nom_table;
- Modifier une table (ajouter une colonne) : ALTER TABLE nom_table ADD colonne type;
- Supprimer une table : DROP TABLE nom_table;

3. Manipulation des Données

- Insérer des données : INSERT INTO nom_table (colonne1, colonne2) VALUES (valeur1, valeur2);
- Afficher les données : SELECT * FROM nom_table;
- Mettre à jour des données : UPDATE nom_table SET colonne1 = valeur1 WHERE condition;
- Supprimer des données : DELETE FROM nom_table WHERE condition;

4. Requêtes Avancées

- Filtrer les résultats : SELECT * FROM nom_table WHERE condition;
- Trier les résultats : SELECT * FROM nom_table ORDER BY colonne ASC|DESC;
- Joindre des tables : SELECT * FROM table1 JOIN table2 ON table1.colonne = table2.colonne;
- Groupes de données : SELECT colonne, COUNT(*) FROM nom_table GROUP BY colonne;

5. Gestion des Utilisateurs et Permissions

- Créer un utilisateur : CREATE USER 'utilisateur'@'hôte' IDENTIFIED BY 'motdepasse';
- Accorder des permissions : GRANT ALL PRIVILEGES ON nom_base.* TO 'utilisateur'@'hôte';
- Afficher les utilisateurs : SELECT User, Host FROM mysql.user;
- Supprimer un utilisateur : DROP USER 'utilisateur'@'hôte';

6. Sauvegarde et Restauration

- Sauvegarder une base de données : mysqldump -u utilisateur -p nom_base > sauvegarde.sql
- Restaurer une base de données : mysql -u utilisateur -p nom_base < sauvegarde.sql